

ACTIVITAT AVALUABLE AC3**Mòdul:** MP12- Projectes**UF:** UF1**Professor:** Albert Guardiola, Marc Callejón**Data límit d'entrega:** 26/10/2025 23:59**Mètode d'entrega:** Per mitjà del Clickedu de l'assignatura. Les activitats entregades més enllà de la data límit només podran obtenir una nota de 5.**Instruccions:** S'ha d'entregar **un únic fitxer comprimit** amb el nom:***MP12-PROJ1-ENTR3.zip/rar***

Es valorarà la presentació.

Tasca 1. Desenvolupar el codi de l'API especificada, en llenguatge C#. El codi de l'API ha de complir els següents requisits:

Requisits funcionals:

-Adequar-se completament a l'especificació elaborada a l' ENTREGA 1. Qualsevol desviació respecte de l'especificació ha de documentar-se a la Tasca 2.

Requisits tecnològics:

-El codi de l'API ha d'estar modularitzat, com a mínim, de la següent manera.

- Mòdul principal: instanciació i configuració (incloses les rutes) del servidor (Controller)
- Mòdul de base de dades: crides a la base de dades.
- Mòdul d'autenticació: gestió de l'autenticació d'usuaris.

En el nostre cas l'estructura principal de l'API queda així:

 bin		17/10/2025 16:29	Carpeta de archivos	
 Controladores		17/10/2025 16:45	Carpeta de archivos	
 Data		17/10/2025 16:41	Carpeta de archivos	
 Modelos		17/10/2025 16:32	Carpeta de archivos	
 obj		17/10/2025 18:07	Carpeta de archivos	
 Properties		17/10/2025 16:27	Carpeta de archivos	
 appsettings.Development.json		17/10/2025 16:26	Archivo de origen ...	1 KB
 appsettings.json		17/10/2025 17:18	Archivo de origen ...	1 KB
 F1Api.csproj		17/10/2025 16:28	Archivo de origen ...	1 KB
 F1Api.http		17/10/2025 16:26	Archivo HTTP	1 KB
 F1Api.sln		17/10/2025 16:28	Archivo SLN	2 KB
 FM1.db		17/10/2025 17:57	Data Base File	188 KB
 Program.cs		17/10/2025 17:18	Archivo de origen ...	1 KB

Com es pot observar dins del projecte tenim diferents carpetes , en les que destaquen Modelos, Controladores, FM1.db , program.cs y appsetting.json.

Models

La carpeta Modelos o Models en el projecte de C# s'encarrega de definir les estructures de dades principals de l'aplicació. És a dir, aquí es defineix quina informació maneja l' API i com es representa. En el nostre cas hem seguit el mateix model que vam decidir al principi de tot en l'entrega 1

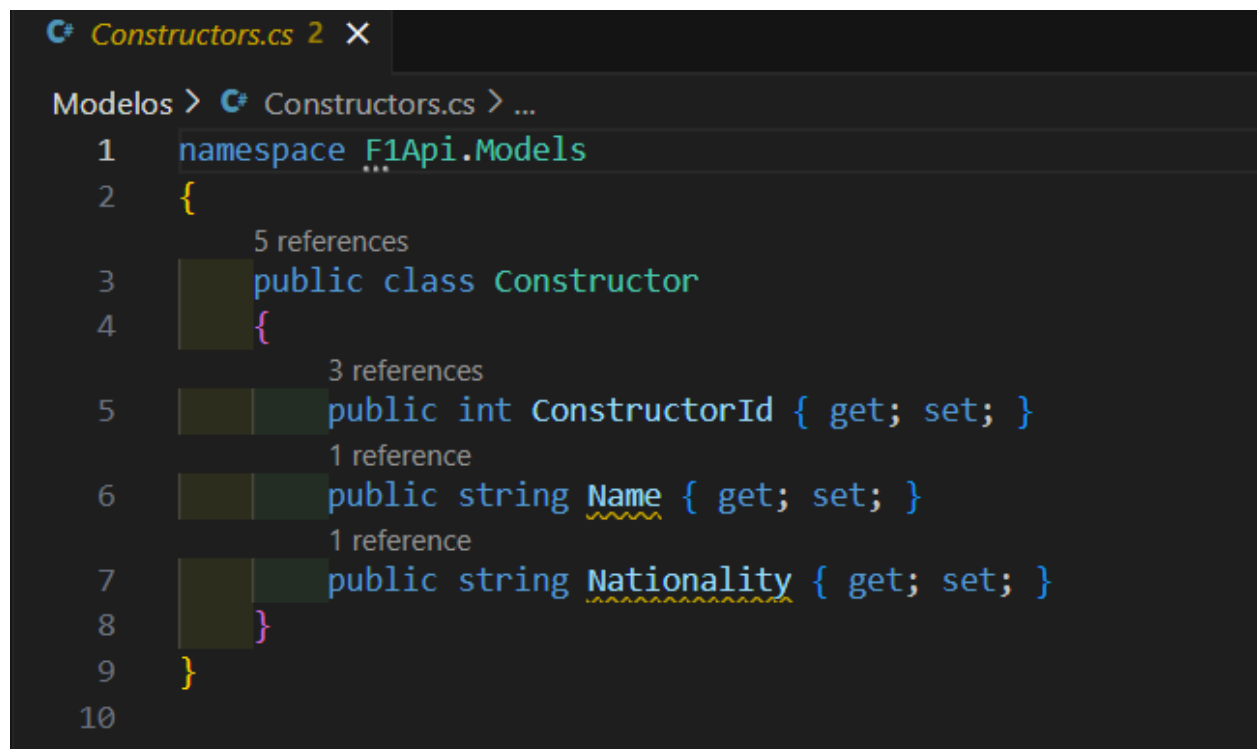
En Modelos tenim les següents estructures:

Circuits.cs

```
C# Circuits.cs 3 X
Modelos > C# Circuits.cs > Circuit
1 namespace F1Api.Models
2 {
3     5 references
4     public class Circuit
5     {
6         3 references
7         public int CircuitId { get; set; }
8         1 reference
9         public string Name { get; set; }
10        1 reference
11        public string Location { get; set; }
12        1 reference
13        public string Country { get; set; }
14        1 reference
15        public double Lat { get; set; }
16        1 reference
17        public double Lng { get; set; }
18    }
19 }
```

```
public class Circuit  
  
{  
  
    public int CircuitId { get; set; }  
  
    public string Name { get; set; }  
  
    public string Location { get; set; }  
  
    public string Country { get; set; }  
  
    public double lat { get; set; }  
  
    public double Lng { get; set; }  
  
}
```

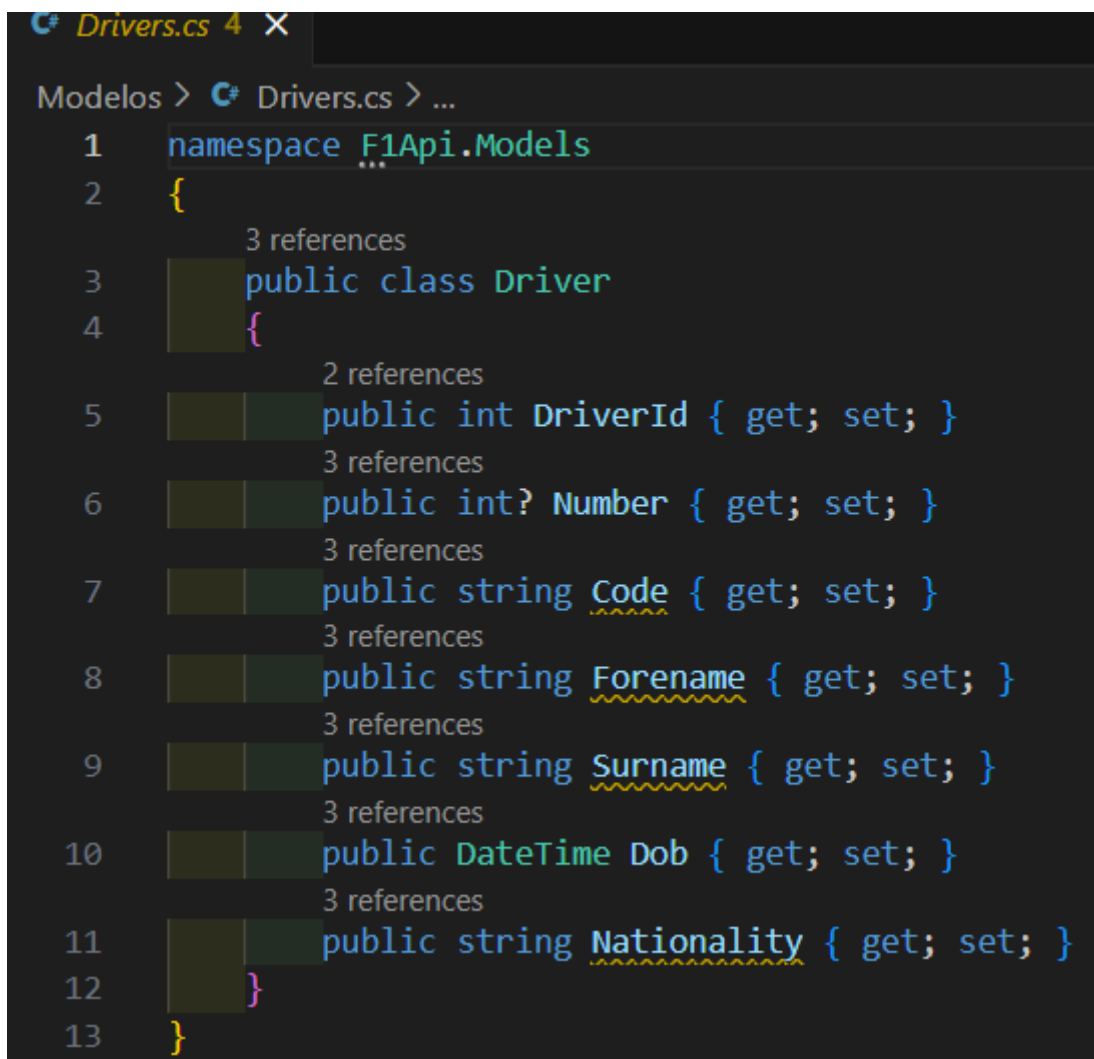
Constructor.cs



```
C# Constructors.cs 2 X  
Modelos > C# Constructors.cs > ...  
1 namespace F1Api.Models  
2 {  
3     5 references  
4     public class Constructor  
5     {  
6         3 references  
7         public int ConstructorId { get; set; }  
8         1 reference  
9         public string Name { get; set; }  
10        1 reference  
        public string Nationality { get; set; }  
    }  
}
```

public class Constructor

```
{  
  
    public int ConstructorId { get; set; }  
  
    public string Name { get; set; }  
  
    public string Nationality { get; set; }  
  
}
```

Drivers.cs

```
# Drivers.cs 4 X  
Modelos > C# Drivers.cs > ...  
1 namespace F1Api.Models  
2 {  
3     3 references  
4     public class Driver  
5     {  
6         2 references  
7         public int DriverId { get; set; }  
8         3 references  
9         public int? Number { get; set; }  
10        3 references  
11        public string Code { get; set; }  
12        3 references  
13        public string Forename { get; set; }  
14        3 references  
15        public string Surname { get; set; }  
16        3 references  
17        public DateTime Dob { get; set; }  
18        3 references  
19        public string Nationality { get; set; }  
20    }  
21 }
```

```
public class Driver  
  
{  
  
    public int DriverId { get; set; }  
  
    public int Number { get; set; }  
  
    public int Code { get; set; }  
  
    public string Forename { get; set; }  
  
    public string Surname { get; set; }  
  
    public DateTime dob { get; set; }  
  
    public string Nationality { get; set; }  
  
}
```

Hem decidit canviar el tipus de dades d'int a string a la propietat Code del model Drivers perquè, al dataset de Fórmula 1, aquest camp representa un codi únic de pilot format per lletres com ara "HAM" per a Lewis Hamilton o "VEURE" per a Max Verstappen. Com que aquests codis no són valors numèrics, considerem utilitzar un tipus string millor i adequadament.

FM1.db

Per al nostre projecte de l'API de Fórmula 1, hem creat la base de dades FM1.db utilitzant DB Browser for SQLite de la manera següent:

Teníem les dades de circuits, pilots i escuderies en fitxers CSV. Vam obrir DB Browser for SQLite, vam crear una nova base de dades anomenada FM1.db i vam utilitzar l'opció de importar CSV a taula per a cada fitxer. Cada CSV es va convertir en una taula dins de la base de dades:

- Circuits amb columnes: CircuitId, Name, Location, Country, Lat, Lng.*
- Drivers amb columnes: DriverId, Number, Code, Forename, Surname, Dob, Nationality.*
- Constructors amb columnes: ConstructorId, Name, Nationality.*

En importar els CSV, vam assignar tipus de dades adequats:

- int per a identificadors (CircuitId, DriverId, ConstructorId) i números de pilot (Number).*
- string per a text (Name, Location, Country, Code, Forename, Surname, Nationality).*
- double per a coordenades (Lat, Lng).*

Després d'importar els CSV, vam comprovar que totes les taules i columnes s'havien creat correctament i que les dades estiguessin completes i consistents.

Un cop creada la base de dades, la vam utilitzar al projecte F1Api, configurant la cadena de connexió cap a FM1.db. Això ens permet realitzar operacions CRUD des de l'API utilitzant Entity Framework Core.

TABLES		circuitId	name	location	country	lat	lng
> circuits		1	Albert Park Grand Prix Circuit	Melbourne	Australia	-37.8497	144.968
> constructors (1)		2	Sepang International Circuit	Kuala Lumpur	Malaysia	2.76083	101.738
> drivers		3	Bahrain International Circuit	Sakhir	Bahrain	26.0325	50.5106
		4	Circuit de Barcelona-Catalunya	Montmeló	Spain	41.57	2.26111
		5	Istanbul Park	Istanbul	Turkey	40.9517	29.405
		6	Circuit de Monaco	Monte-Carlo	Monaco	43.7347	7.42056
		7	Circuit Gilles Villeneuve	Montreal	Canada	45.5	-73.5228
		8	Circuit de Nevers Magny-Cours	Magny Cours	France	46.8642	3.16361
		9	Silverstone Circuit	Silverstone	UK	52.0786	-1.01694
		10	Hockenheimring	Hockenheim	Germany	49.3278	8.56583
		11	Hungaroring	Budapest	Hungary	47.5789	19.2486
		12	Valencia Street Circuit	Valencia	Spain	39.4589	-0.331667
		13	Circuit de Spa-Francorchamps	Spa	Belgium	50.4372	5.97139
		14	Autodromo Nazionale di Monza	Monza	Italy	45.6156	9.28111
		15	Marina Bay Street Circuit	Marina Bay	Singapore	1.2914	103.864
		16	Fuji Speedway	Oyama	Japan	35.3717	138.927
		17	Shanghai International Circuit	Shanghai	China	31.3389	121.22
		18	Autódromo José Carlos Pace	So Paulo	Brazil	-23.7036	-46.6997
		19	Indianapolis Motor Speedway	Indianapolis	USA	39.795	-86.2347
		20	Nürburgring	Nürburg	Germany	50.3356	6.9475
		21	Autodromo Enzo e Dino Ferrari	Imola	Italy	44.3439	11.7167
		22	Suzuka Circuit	Suzuka	Japan	34.8431	136.541
	+	75	A1-Ring	Spielburg	Austria	47.2197	14.7647

Rows: 208

	constr...	name	nationality
	  	  	  
	Filte   	Filte   	Filte   
1	1	McLaren	British
2	2	BMW Sauber	German
3	3	Williams	British
4	4	Renault	French
5	5	Toro Rosso	Italian
6	6	Ferrari	Italian
7	7	Toyota	Japanese
8	8	Super Aguri	Japanese
9	9	Red Bull	Austrian
10	10	Force India	Indian
11	11	Honda	Japanese
12	12	Spyker	Dutch
13	13	MF1	Russian
14	14	Spyker MF1	Dutch
15	15	Sauber	Swiss
16	16	BAR	British
17	17	Jordan	Irish
18	18	Minardi	Italian
19	19	Jaguar	British
20	20	Prost	French
21	21	Arrows	British
22	22	Benetton	Italian
23	23	Brawn	British
+ 209			

	driverId 	number 	code 	forename 	surname 	dob 	nationality 
							
1	1	44	HAM	Lewis	Hamilton	07/01/1985	British
2	2	NULL	HEI	Nick	Heidfeld	10/05/1977	German
3	3	6	ROS	Nico	Rosberg	27/06/1985	German
4	4	14	ALO	Fernando	Alonso	29/07/1981	Spanish
5	5	NULL	KOV	Heikki	Kovalainen	19/10/1981	Finnish
6	6	NULL	NAK	Kazuki	Nakajima	11/01/1985	Japanese
7	7	NULL	BOU	Šbastien	Bourdais	28/02/1979	French
8	8	7	RAI	Kimi	Räikkönen	17/10/1979	Finnish
9	9	NULL	KUB	Robert	Kubica	07/12/1984	Polish
10	10	NULL	GLO	Timo	Glock	18/03/1982	German
11	11	NULL	SAT	Takuma	Sato	28/01/1977	Japanese
12	12	NULL	PIQ	Nelson	Piquet Jr.	25/07/1985	Brazilian
13	13	19	MAS	Felipe	Massa	25/04/1981	Brazilian
14	14	NULL	COU	David	Coulthard	27/03/1971	British
15	15	NULL	TRU	Jarno	Trulli	13/07/1974	Italian
16	16	99	SUT	Adrian	Sutil	11/01/1983	German
17	17	NULL	WEB	Mark	Webber	27/08/1976	Australian
18	18	22	BUT	Jenson	Button	19/01/1980	British
19	19	NULL	DAV	Anthony	Davidson	18/04/1979	British
20	20	5	VET	Sebastian	Vettel	03/07/1987	German

Appsetting.json

```

1 appsettings.json X
2 appsettings.json > ...
3 {
4   "ConnectionStrings": {
5     "F1Connection": "Data Source=C:\\Users\\mrobe\\OneDrive\\Documentos\\DAW 2\\Proyecto definitivo\\F1Api\\FM1.db"
6   },
7   "Logging": {
8     "LogLevel": {
9       "Default": "Information",
10      "Microsoft.AspNetCore": "Warning"
11    }
12  },
13  "AllowedHosts": "*"
14 }

```

```

{
  "ConnectionStrings": {
    "F1Connection": "Data Source=C:\\Users\\mrobe\\OneDrive\\Documentos\\DAW 2\\Proyecto definitivo\\F1Api\\FM1.db"
  },
}

```

Aquesta secció defineix les cadenes de connexió a la base de dades. En aquest cas, "F1Connection" és el nom de la connexió i "Data Source=..." indica la ruta del fitxer de base de dades SQLite (FM1.db). Cada vegada que el teu codi necessiti connectar-se a la base de dades, farà servir aquest nom per obtenir la cadena de connexió.

```
"Logging": {  
  "LogLevel": {  
    "Default": "Information",  
    "Microsoft.AspNetCore": "Warning"  
  }  
},
```

Aquí es configura el registre de missatges. "Default": "Information" significa que es registren tots els missatges de nivell informació o superior, mentre que "Microsoft.AspNetCore": "Warning" limita els missatges generats pel framework ASP.NET Core només a advertències i errors. Això permet depurar sense omplir els logs amb missatges poc rellevants.

```
"AllowedHosts": "*"
```

Defineix quins dominis poden accedir a l'API. El valor "" permet l'accés des de qualsevol host, cosa útil en desenvolupament; en producció normalment es restringeix a dominis específics per seguretat.*

Program.CS

```
Program.cs X  
F1Api > Program.cs > Program > <top-level-statements-entry-point>  
1 using F1Api.Data;  
2 using Microsoft.EntityFrameworkCore;  
3  
4 var builder = WebApplication.CreateBuilder(args);  
5  
6  
7 builder.Services.AddDbContext<Fm1Context>(options =>  
8     options.UseSqlite(builder.Configuration.GetConnectionString("F1Connection")));  
9  
10 builder.Services.AddControllers();  
11 builder.Services.AddEndpointsApiExplorer();  
12 builder.Services.AddSwaggerGen();  
13  
14 var app = builder.Build();  
15  
16 app.UseSwagger();  
17 app.UseSwaggerUI();  
18  
19 app.UseHttpsRedirection();  
20 app.UseAuthorization();  
21 app.MapControllers();  
22  
23 app.Run();  
24
```

```
using F1Api.Data;  
using Microsoft.EntityFrameworkCore;
```

using F1Api.Data: importa l'espai de noms on es troba la classe Fm1Context, que és el DbContext (és a dir, la connexió i accés a la base de dades).

using Microsoft.EntityFrameworkCore: permet utilitzar Entity Framework Core, la llibreria d'ORM que facilita treballar amb bases de dades en forma d'objectes.

```
var builder = WebApplication.CreateBuilder(args);
```

Això crea un constructor (builder) per configurar serveis i la infraestructura de l'aplicació web. És el primer pas per preparar la nostra API.

```
builder.Services.AddDbContext<Fm1Context>(options =>  
    options.UseSqlite(builder.Configuration.GetConnectionString("F1Connection")));
```

AddDbContext registra la classe Fm1Context com a servei dins de l'aplicació.

UseSqlite indica que la base de dades que farà servir és SQLite.

builder.Configuration.GetConnectionString("F1Connection") obté la cadena de connexió (normalment guardada a appsettings.json) per saber on es troba la base de dades.

```
builder.Services.AddControllers();  
builder.Services.AddEndpointsApiExplorer();  
builder.Services.AddSwaggerGen();
```

AddControllers(): afegeix el suport per utilitzar controladors (les classes que contenen les rutes i endpoints de l'API).

AddEndpointsApiExplorer(): permet explorar automàticament els endpoints de l'API (necessari per Swagger).

AddSwaggerGen(): activa Swagger, una eina que genera documentació interactiva de l'API.

```
var app = builder.Build();
```

Aquí es construeix l'aplicació amb totes les configuracions fetes anteriorment

```
app.UseSwagger();
app.UseSwaggerUI();
```

Activa Swagger i Swagger UI, que permet provar els endpoints des del navegador

```
app.UseHttpsRedirection();
```

Redirigeix tot el tràfic HTTP a HTTPS per més seguretat.

Autenticació

```
3 references
private const string API_KEY = "TU_API_KEY";
```

Per cada persona que vulgui afegir, o eliminar alguna cosa ha de posar un API KEY que en nostre cas es "TU_API_KEY"

Tasca 2. Lliurar un *checklist* dels requisits funcionals especificats (ENTREGA 1) que marqui clarament si s'han implementat o no en el codi lliurat. Els requisits no implementats han de venir acompanyats d'una justificació de per què no s'han implementat.

	CIRCUITS	CONSTRUCTORS	DRIVERS
GET	Implementat*1	Implementat*3	Implementat
POST	Implementat	Implementat	Implementat
PUT	Implementat*2	Implementat	Implementat
DELETE	Implementat	Implementat	Implementat

Ruta /api/circuits

GET → consulta de circuits.

POST → afegir un circuit nou.

PUT → modificar informació d'un circuit.

DELETE → eliminar un circuit existent.

Ruta /api/constructors

GET → consulta escuderías constructors.

POST → afegir una escudería nova.

PUT → modificar informació d'una escudería.

DELETE → eliminar una escudería existent.

Ruta /api/drivers

GET → consulta de pilots.

POST → afegir un pilot nou.

PUT → modificar informació d'un pilot.

DELETE → eliminar un pilot existent.

S'han implementat els mètodes documentats en la Entrega 1.

5. MÈTODE D'AUTENTICACIÓ

En aquesta API, l'usuari podrà realitzar consultes mitjançant el mètode GET sense necessitat d'autenticació.

En canvi, per a la resta de mètodes (POST, DELETE i PUT) serà obligatori disposar i utilitzar una API_KEY vàlida per tal de garantir la seguretat i el control d'accés.

Circuits

GET /api/Circuits

Parameters

Name	Description
circuitid	circuitid
integer(int32)	(query)
name	name
string	(query)
location	location
string	(query)
country	country
string	(query)
lat	lat
number(double)	(query)
lng	lng
number(double)	(query)

POST /api/Drivers

Parameters

Name	Description
API_KEY	API_KEY
string	(header)

Request body

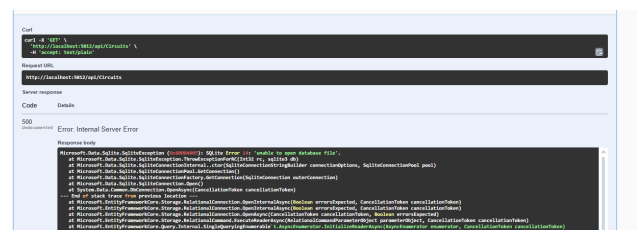
application/json

Example Value | Schema

```
{
  "driverId": 0,
  "number": 0,
  "code": "string",
  "forename": "string",
  "surname": "string",
  "dob": "2025-10-23T15:36:31.896Z",
  "nationality": "string"
}
```

S'ha implementat una API_KEY a tots els mètodes en circuits, drivers, i constructors, excepte el mètode GET.

*1



Error en la direcció de la database.

***2**

Error al igualar la id requerida amb [circuits.Circuits.id](#).

***3**

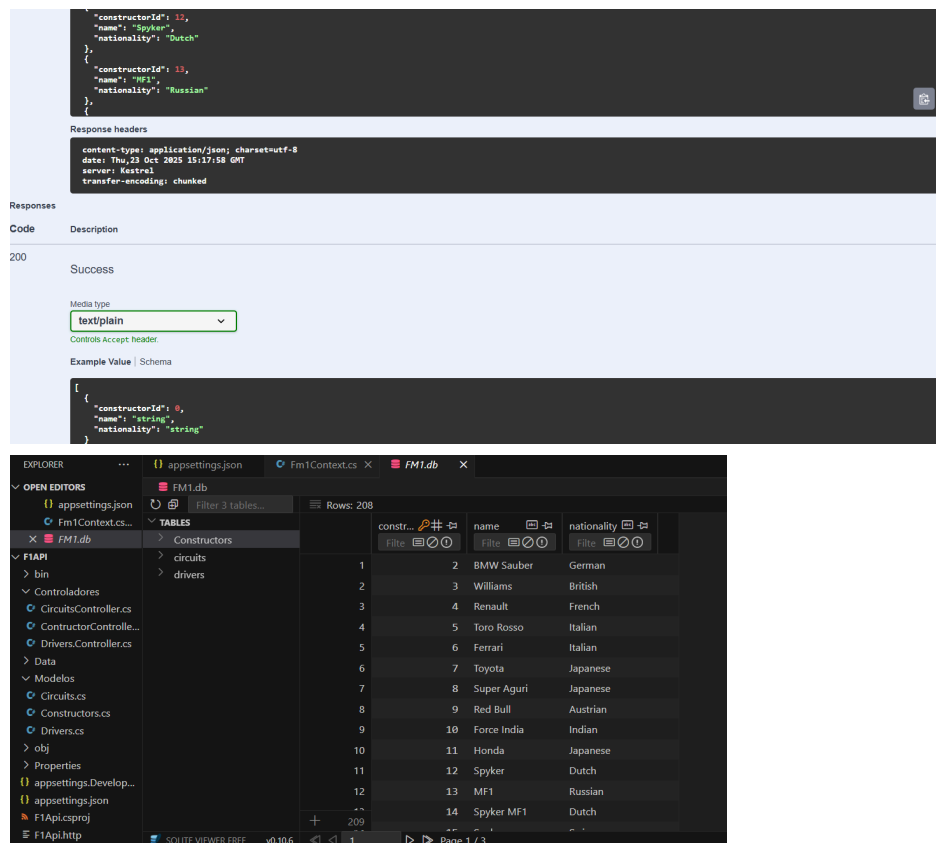
```

Code      Details
500        Error: Internal Server Error
            (sharded)

Response body

Microsoft.Data.Sqlite.SQLiteException (in=80004001): SQLite Error 1: "no such table: Constructors".
    at Microsoft.Data.Sqlite.SQLiteException.ThrowExceptionFor(Sqlite3 rc, sqlite3 db)
    at Microsoft.Data.Sqlite.SQLiteCommand.PrepareAndExecuteStatements(IValueHolder[])
    at Microsoft.Data.Sqlite.SQLiteCommand.GetStatements()>MoveNext()
    at Microsoft.Data.Sqlite.SQLiteCommand.ExecuteReader(CommandBehavior behavior)
    at Microsoft.Data.Sqlite.SQLiteCommand.ExecuteReaderAsync(CommandBehavior behavior, CancellationToken cancellationToken)
    at Microsoft.Data.Sqlite.SQLiteCommand.ExecuteReaderAsync(CancellationToken cancellationToken)
    at Microsoft.EntityFrameworkCore.Storage.RelationalCommand.ExecuteReaderAsync(RelationalCommandParameterObject parameterObject, CancellationToken cancellationToken)
    at Microsoft.EntityFrameworkCore.Storage.RelationalCommand.ExecuteReaderAsync(RelationalCommandParameterObject parameterObject, CancellationToken cancellationToken)
    at Microsoft.EntityFrameworkCore.Query.Internal.SingleQueryingEnumerable`1.AsyncEnumerator.InitializeReaderAsync(AsyncEnumerator enumerator, CancellationToken cancellationToken)
    at Microsoft.EntityFrameworkCore.Query.Internal.SingleQueryingEnumerable`1.AsyncEnumerator.MoveNextAsync()
    at Microsoft.Data.Sqlite.SQLiteCommand.ExecuteReaderAsync(CancellationToken cancellationToken)
    at Microsoft.EntityFrameworkCore.EntityFrameworkQueryableExtensions.ToListAsync(IEnumerable<IQueryable> source, CancellationToken cancellationToken)
    at Microsoft.EntityFrameworkCore.EntityFrameworkQueryableExtensions.ToListAsync(IEnumerable<IQueryable> source, CancellationToken cancellationToken)
    at IPagedList.ConstructorController.GetConstructors(Mutable < constructorId, String name, String nationality> in C:\Users\jwoda\OneDrive\Documents\BMW 2\Projects\Projecto definitivo\IPagedList\Controllers\ConstructorController.cs:line 28
    at lambda_method(Closure, Object)
    at Microsoft.AspNetCore.Mvc.Infrastructure.ActionMethodExecutor.AwaitableTypeActionResultExecutor.Execute(HttpContext context, ActionContext actionContext, IActionResultTypeMapper mapper, ObjectMethodExecutor executor, Object controller, Object[] arguments)
    at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.InvokeActionMethodAsync_Awaited[12](ControllerActionInvoker invoker, ValueTask< actionResult> valueTask)
    at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.InvokeActionMethodAsync_Awaited[10](ControllerActionInvoker invoker, Task lastTask, State next, Scope scope, Object ct, ValueTask< bool> isDisposable)
    at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.Rethrow(HttpContext context, ActionContext actionContext)
    at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.Next(State next, Scope scope, Object state, Boolean isCompleted)
    at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.InvokeInnerFilterAsync()
    --- End of stack trace from previous location ---
    at Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.CinkonfilterPipelineAsync_Awaited[10](ResourceInvoker invoker, Task lastTask, State next, Scope scope, Object state, Boolean isCompleted)

```



The top screenshot shows a REST client interface with a successful response (200) for a GET request. The response body is a JSON array of constructor objects. The bottom screenshot shows a database browser (FM1.db) with a table named 'Constructors' containing 14 rows of data, including columns for constructor ID, name, and nationality.

Error en el nomenclado de la ruta Constructors. En la database, es va nombrar la ruta Constructors com Constructors (1). Es va corregir al DB Browser.

Tasca 3. Allotjar l'API a un servidor de C#

Error HTTP 500.19 - Internal Server Error

No se puede obtener acceso a la página solicitada porque los datos de configuración relacionados de la página no son válidos.

Información detallada de error:

Módulo	IIS Web Core	Dirección URL solicitada	http://localhost:8080/
Notificación	Desconocido	Ruta de acceso física	
Controlador	No determinado aún	Método de inicio de sesión	n
Código de error	0x8007000d	Usuario de inicio de sesión	No determinado aún
Error de configuración	n		
Archivo de configuración	\\?C:\VF1Api\web.config		

Origen de configuración:

```

-1:
0:

```

Más información:

Se produce este error cuando surge algún problema al leer el archivo de configuración del servidor web o la aplicación web. En algunos casos, los registros de eventos pueden contener más información sobre el motivo de este error.

[Ver más información >](#)

El error HTTP 500.19 indica un fallo en la lectura o interpretación de la configuración del servidor web. En el caso de aplicaciones ASP.NET Core, IIS utiliza el `AspNetCoreModuleV2` para iniciar la aplicación, y si este módulo no está registrado correctamente, la aplicación no puede arrancar.

Durante el proceso de implementación se observó que aunque el Hosting Bundle de .NET 8 estaba instalado, IIS no detectaba los módulos de ASP.NET Core en la sección de Módulos del servidor. Esto impide que IIS ejecute la DLL de la aplicación (`F1Api.dll`), generando el error 500.19.

Link al tutorial d'exemple:

[Working with SQL Lite Database in Asp.NET Core Web API](#)

